

LABORATORIO 08

¿Y quién mueve los datos?

Sumar un broker en caliente.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers y uno más

Duración estimada · 50 minutos · necesitas dos terminales

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a haber metido **un cuarto servidor a un clúster que está atendiendo**, y le vas a haber movido carga sin detener nada.

Y antes de eso vas a ver algo que a mucha gente le cuesta caro. **El servidor nuevo se va a quedar ahí, encendido, sano, costando dinero, y vacío**, hasta que alguien se lo ordene.

LO QUE ESTE LABORATORIO DEMUESTRA

Se le puede agregar un servidor a un clúster que está atendiendo y moverle carga sin detener nada.

Y LA LETRA CHICA, QUE ES LA MITAD DEL VALOR

«Sin detener nada» no significa «sin que nadie lo note». No se pierde un mensaje, y la cola de latencia se entera.

UN AVISO SOBRE EL COMANDO DE HOY

`kafka-reassign-partitions` es **el comando más peligroso del curso**. Copia gigabytes entre servidores mientras el clúster atiende, y compite por la misma red y los mismos discos que tus clientes.

Por eso te deja **un plan de vuelta y un freno**. Este laboratorio trata sobre esos dos, no sobre el comando.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 8 GB de RAM	Hoy son cuatro brokers, no tres. Ajustes de Docker Desktop, pestaña <i>Resources</i>
Dos terminales de Git Bash	Una para el tráfico de fondo y otra para trabajar
Las dos en la carpeta del laboratorio	<code>cd Capitulo_3/lab-08-brokers-en-caliente</code>
Los puertos 9092, 9093, 9094, 9095 y 8090 libres	El cuarto broker usa el 9095

CÓMO VAMOS A LLAMAR A LAS TERMINALES

Terminal	Para qué
P , la del productor	Aquí corre el tráfico de fondo durante todo el laboratorio. No la vas a poder usar para otra cosa
T , la de trabajo	Desde aquí se agrega el broker, se reasigna y se consulta todo

El caso

Viene el Cyber. El equipo de **NovaTech Logistics** decide sumar un cuarto broker al clúster para aguantar el pico. Levantan el servidor, lo unen al clúster, confirman que aparece en la lista.

Y no pasa nada.

El tráfico sigue repartido entre los tres de siempre. El servidor nuevo está encendido, sano, costando dinero, y vacío. Alguien pregunta cuándo empieza a recibir carga, y la respuesta honesta es **nunca, si nadie se lo ordena**.

El problema no es Kafka. El problema es una confusión que se paga cara. **Agregar capacidad y usar capacidad son dos operaciones distintas**, y la segunda mueve datos por la red mientras tus clientes están conectados.

LA METÁFORA · ENTRA UN MOZO NUEVO A MITAD DE SERVICIO

Kafka sigue siendo **el restaurante**. El mozo es el broker, el tipo de comanda es el tópico, los sectores del salón son las particiones, las libretas de respaldo son las réplicas.

Hoy **entra un mozo nuevo a mitad de servicio**. Ponerle el uniforme y presentarlo al equipo es una cosa. Que le toque atender mesas es otra, porque alguien tiene que **reasignarle sectores del salón**, y eso significa que un mozo que ya venía atendiendo la mesa 7 se la pasa al nuevo, con los clientes sentados y a mitad del plato.

En el restaurante	En Kafka
Entra el mozo nuevo y se presenta	El broker 4 arranca y se une al clúster
Sigue parado en la puerta	El broker 4 no tiene ninguna partición
Se le reparten sectores del salón	<code>kafka-reassign-partitions</code>
El traspaso de cada mesa, con el cliente ahí	La copia de cada réplica, con el productor escribiendo
El papelito con quién atendía qué antes	El plan de vuelta
«Vayan despacio, que hay clientes»	El throttle

Las dos últimas filas son las que nadie anota, y son la segunda mitad de este laboratorio.

Levantar el clúster y tomar la foto de «antes»

Qué vamos a hacer

Un clúster de tres brokers con un tópicos de 6 particiones y factor de replicación 3. Cada partición vive en tres brokers, así que hay **18 réplicas** repartidas entre tres servidores.

Antes de tocar nada hay que fotografiar dónde está cada una. **Contra esa foto vas a comparar todo lo demás.**

Los comandos, en la terminal T

```
cd Capitulo_3/lab-08-brokers-en-caliente
chmod +x bin/*.sh kafka-cli/*.sh
bin/start-lab.sh
kafka-cli/describe-topic.sh novatech.lab08.pedidos
```

bin/start-lab.sh — levanta los tres brokers y la interfaz web. **El cuarto no lo levanta**, ese es tu trabajo en el paso 3

kafka-cli/describe-topic.sh — envoltorio del curso. Por dentro llama a `kafka-topics --describe`

novatech.lab08.pedidos — el tópicos del laboratorio, que el `start-lab.sh` deja creado

LA FOTO DE «ANTES» · ESTO SALIÓ EN UNA CORRIDA REAL

```
Topic: novatech.lab08.pedidos PartitionCount: 6 ReplicationFactor: 3
Configs: min.insync.replicas=2
Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Partition: 2 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
Partition: 3 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
Partition: 4 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
Partition: 5 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
```

Cómo se lee

Columna	Qué dice
<code>Replicas: 3,1,2</code>	En qué brokers vive esta partición. El primero de la lista es el líder preferido
<code>Leader: 3</code>	Quién atiende ahora las lecturas y escrituras. Los otros dos solo copian
<code>Isr: 3,1,2</code>	Las copias que están al día . Si una se atrasa, sale de aquí

VAS BIEN SI...

Salen **seis líneas de partición**, y en toda la columna `Replicas` **solo aparecen el 1, el 2 y el 3**.

A ti los números te van a salir repartidos de otra manera, porque Kafka decide el reparto al crear el tópico. Lo que no cambia es que no hay ningún 4.

SI SALE `TIMEOUT: SOLO 2/3 BROKERS ESTÁN OPERATIVOS`

Es memoria. **Este laboratorio va a levantar cuatro brokers**, así que necesita más que los anteriores. Sube Docker Desktop a 8 GB en *Resources* y ejecuta `bin/start-lab.sh` otra vez.

SI EL TÓPICO NO EXISTE

El `start-lab.sh` no llegó a crearlo. Créalo tú y sigue:

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --create --topic novatech.lab08.pedidos \  
  --partitions 6 --replication-factor 3
```

Poner tráfico, y que dure lo suficiente

Qué vamos a hacer

Para poder afirmar «sin detener nada» hace falta que **haya algo que detener**. Antes de tocar el clúster, ponemos un productor a escribir y lo dejamos corriendo todo el laboratorio.

Y AQUÍ HAY UNA TRAMPA QUE ARRUINA EL LABORATORIO EN SILENCIO

El envoltorio `kafka-cli/produce-sample.sh` escribe **a toda velocidad**, sin tope. En una máquina rápida, sus 300 000 mensajes se acaban **en dos segundos**.

Si eso pasa, cuando llegues a reasignar **ya no hay tráfico**, y la demostración no demuestra nada. No falla ni avisa. Simplemente no hay nada que mirar.

La salida es ponerle un tope al caudal, para que la duración sea la misma en cualquier máquina. Por eso aquí va el comando crudo.

El comando, en la terminal P

```
docker exec kafka-broker-1 kafka-producer-perf-test \
  --topic novatech.lab08.pedidos \
  --num-records 300000 --record-size 256 \
  --throughput 5000 \
  --producer-props bootstrap.servers=kafka-broker-1:29092
```

kafka-producer-perf-test — la herramienta oficial de carga, la misma del laboratorio 07. Aquí no la usamos para medir, la usamos para **generar tráfico realista**

--num-records 300000 — cuántos mensajes en total

--record-size 256 — cuántos bytes pesa cada uno

--throughput 5000 — **el tope, y es la clave de este paso**. 5 000 mensajes por segundo. 300 000 dividido por 5 000 son **60 segundos de tráfico**, en cualquier máquina

--producer-props bootstrap.servers=... — a qué clúster escribe

LO QUE VAS A VER MIENTRAS CORRE

```
24998 records sent, 4999.6 records/sec (1.22 MB/sec), 8.4 ms avg latency, 235.0 ms max latency.
25002 records sent, 5000.4 records/sec (1.22 MB/sec), 6.9 ms avg latency, 42.0 ms max latency.
...
```

Una línea cada cinco segundos. **Déjalo corriendo y vuelve a la terminal T.** Tienes 60 segundos, que alcanzan de sobra para los pasos 3 y 4.

VAS BIEN SI...

La terminal P **sigue imprimiendo líneas** y el `records/sec` ronda los 5 000.

Comprueba que sigue viva antes de cada paso. Si terminó, vuelve a lanzarla. Sin tráfico, los pasos 4 y 7 no muestran nada.

SI EL PRODUCTOR TERMINA ANTES DE QUE LLEGUES AL PASO 4

Te pasó porque los pasos anteriores te tomaron más de un minuto, que es lo normal la primera vez.

Relánzalo con más mensajes y tendrás cinco minutos:

```
docker exec kafka-broker-1 kafka-producer-perf-test \  
  --topic novatech.lab08.pedidos \  
  --num-records 1500000 --record-size 256 \  
  --throughput 5000 \  
  --producer-props bootstrap.servers=kafka-broker-1:29092
```

Agregar el broker 4, y ver que no pasa nada

Qué vamos a hacer

Se levanta un cuarto broker y se une al clúster. **Fíjate en lo que no cambia**, que es todo el punto del paso.

Los comandos, en la terminal T

```
kafka-cli/add-broker.sh
kafka-cli/list-brokers.sh
```

kafka-cli/add-broker.sh — levanta el contenedor `kafka-broker-4`, que el compose trae declarado bajo un perfil aparte. **Tarda unos 11 segundos**

kafka-cli/list-brokers.sh — le pregunta al clúster quién está y en qué papel

EL FINAL DE `LIST-BROKERS.SH` · RECORTADO A LO ANCHO

```
LeaderId:          2
LeaderEpoch:      1
CurrentVoters:     [{"id": 1, "endpoints": ["CONTROLLER://kafka-broker-1:39092"]},
                    {"id": 2, "endpoints": ["CONTROLLER://kafka-broker-2:39093"]},
                    {"id": 3, "endpoints": ["CONTROLLER://kafka-broker-3:39094"]}]]
CurrentObservers:  [{"id": 4, "directoryId": "X9GHyPeS16rwsSyDOYZYyQ"}]]
```

Cómo se lee, y son dos cosas

CONCEPTO · QUÓRUM DE CONTROLADORES

El grupo de nodos que decide quién es líder de qué. **Se dimensiona una vez, con tres o cinco, y se deja quieto**. Escalar brokers y escalar el quórum son decisiones distintas, y mezclarlas es una fuente clásica de problemas.

Uno. El quórum no se tocó. Los *voters* siguen siendo 1, 2 y 3. El broker 4 entra como **observer**, o sea que aporta capacidad de almacenamiento y de cómputo pero **no vota** en las decisiones del clúster.

Dos. El broker 4 está vacío. Vuelve a mirar la foto:

```
kafka-cli/describe-topic.sh novatech.lab08.pedidos
```

```

Partition: 0  Leader: 3  Replicas: 3,1,2  Isr: 3,1,2
Partition: 1  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3
Partition: 2  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
Partition: 3  Leader: 2  Replicas: 2,3,1  Isr: 2,3,1
Partition: 4  Leader: 3  Replicas: 3,1,2  Isr: 3,1,2
Partition: 5  Leader: 1  Replicas: 1,2,3  Isr: 1,2,3

```

ESTE ES EL PUNTO DEL LABORATORIO

Ni un 4 en toda la columna `Replicas` . Las 18 réplicas siguen exactamente donde estaban. Es la misma foto del paso 1, sin una sola diferencia.

El servidor está encendido, sano, unido al clúster, contando en tu factura. Y no está haciendo nada. **Agregar capacidad no mueve datos.** Alguien tiene que ordenarlo.

VAS BIEN SI...

En `CurrentVoters` hay **tres** identificadores y en `CurrentObservers` está el **4**.

Y la foto del tópico **no cambió respecto del paso 1**. Compáralas línea por línea.

SI `ADD-BROKER.SH` FALLA POR MEMORIA

El cuarto broker es el que empuja el consumo por encima de lo que Docker tiene asignado. Sube a 8 GB en *Resources*. **No hace falta rehacer el laboratorio**, Docker reinicia y los volúmenes siguen ahí. Vuelve a lanzar `bin/start-lab.sh` y repite desde el paso 2.

SI EL BROKER 4 APARECE EN `CURRENTVOTERS`

No debería, y si pasa es que el clúster se levantó con una configuración distinta a la del laboratorio. Baja todo con `bin/stop-lab.sh` , vuelve a `bin/start-lab.sh` y repite. **El quórum de este lab es siempre de tres.**

Reasignar las réplicas

Qué vamos a hacer

CONCEPTO · REASIGNAR

Cambiar **en qué brokers vive** cada réplica. **No cambia el número de particiones, no cambia el orden y no toca un solo mensaje.** Lo único que hace es copiar datos de un servidor a otro.

El comando trabaja en **tres fases separadas a propósito.**

Fase	Qué hace	¿Mueve datos?
<code>--generate</code>	Propone un plan y lo imprime	No
<code>--execute</code>	Aplica el plan. Empieza la copia	Sí
<code>--verify</code>	Pregunta cómo va, y al terminar limpia	No

Están separadas porque **el plan que Kafka propone no tiene por qué gustarte.** Entre `--generate` y `--execute` hay un hueco deliberado para que un humano lea lo que va a pasar.

Antes de ejecutar, comprueba el tráfico

Mira la terminal P. Si dejó de imprimir líneas, relanza el productor del paso 2 antes de seguir. Lo que viene solo tiene gracia con tráfico encima.

El comando, en la terminal T

```
kafka-cli/reassign-partitions.sh novatech.lab08.pedidos 1,2,3,4 8000000
```

novatech.lab08.pedidos — qué tópico se reparte

1,2,3,4 — entre qué brokers. **Los que no estén aquí quedan fuera**, y esa es la manera de vaciar un broker

8000000 — el throttle. **8 MB por segundo de techo para la copia**, para que no se coma el ancho de banda de tus clientes

LOS TRES COMANDOS REALES, LOS QUE VAS A ESCRIBIR EN SUNAT

El envoltorio hace las tres fases seguidas. En un servidor las escribes tú, una por una:

```
kafka-reassign-partitions --bootstrap-server kafka-broker-1:29092 \
  --topics-to-move-json-file topics.json --broker-list 1,2,3,4 --generate

kafka-reassign-partitions --bootstrap-server kafka-broker-1:29092 \
  --reassignment-json-file plan.json --execute --throttle 8000000

kafka-reassign-partitions --bootstrap-server kafka-broker-1:29092 \
  --reassignment-json-file plan.json --verify
```

LO QUE VAS A VER · RECORTADO, EL JSON COMPLETO ES MUY LARGO

```
[Reasignación] novatech.lab08.pedidos → brokers 1,2,3,4 (throttle 8000000 B/s)
[1/3] Generando plan...
Plan propuesto:
{"version":1,"partitions":[{"topic":"novatech.lab08.pedidos","partition":0,
"replicas":[2,3,4],...}]}

Plan de VUELTA guardado en /tmp/lab08-rollback-novatech.lab08.pedidos.json

[2/3] Ejecutando...
Current partition replica assignment
{"version":1,"partitions":[{"topic":"novatech.lab08.pedidos","partition":0,
"replicas":[1,2,3],...}]}

Save this to use as the --reassignment-json-file option during rollback
Warning: You must run --verify periodically, until the reassignment completes,
to ensure the throttle is removed.
The inter-broker throttle limit was set to 8000000 B/s
Successfully started partition reassignments for novatech.lab08.pedidos-0,...

[3/3] Verificando hasta que termine...
  copiando... 4 de 6 partición(es) pendiente(s) (intento 1/60)
Status of partition reassignment:
Reassignment of partition novatech.lab08.pedidos-0 is completed.
...
Clearing broker-level throttles on brokers 1,2,3,4
Clearing topic-level throttles on topic novatech.lab08.pedidos
✓ Reasignación completa
```

La operación completa tardó **12 segundos** en la corrida medida.

Ahora mira dónde quedaron las réplicas

```
kafka-cli/describe-topic.sh novatech.lab08.pedidos
```

```
Partition: 0 Leader: 3 Replicas: 2,3,4 Isr: 2,3,4
Partition: 1 Leader: 1 Replicas: 3,4,1 Isr: 1,3,4
Partition: 2 Leader: 2 Replicas: 4,1,2 Isr: 1,2,4
Partition: 3 Leader: 3 Replicas: 1,2,3 Isr: 1,2,3
Partition: 4 Leader: 1 Replicas: 4,3,1 Isr: 1,3,4
Partition: 5 Leader: 1 Replicas: 3,1,2 Isr: 1,2,3
```

El 4 aparece ahora en cuatro de las seis particiones, y en dos de ellas es el **primero** de la lista, o sea el líder preferido.

SER LÍDER PREFERIDO NO ES SER LÍDER TODAVÍA

En la partición 2 las réplicas son 4,1,2 pero el `Leader` sigue siendo el 2. **Kafka no le quita el liderazgo a nadie solo porque cambió el orden de la lista.** El traspaso ocurre en la siguiente elección de líder preferido, que corre sola cada cierto tiempo. En la corrida medida, dos particiones ya tenían al 4 de líder y en otra corrida ninguna.

VAS BIEN SI...

El 4 aparece en la columna `Replicas` de varias particiones, y en `Isr` también, que significa que ya está al día.

A ti te puede tocar en tres, en cuatro o en cinco particiones. Lo que no puede pasar es que no aparezca en ninguna.

SI LA REASIGNACIÓN TERMINA INSTANTÁNEAMENTE

El tópico tiene muy pocos datos, así que no hay nada que copiar. Es lo esperado si te saltaste el paso 2. **Produce volumen primero** o no vas a ver nada.

SI DICE `STILL IN PROGRESS` Y SE QUEDA AHÍ

Es correcto. La copia no ha terminado. El envoltorio reintenta hasta 60 veces. Si agotara los 60 intentos, mira los logs con `docker logs kafka-broker-4`, pero con el volumen de este lab no debería pasar.

El plan de vuelta, que se va con el scroll

Qué vamos a hacer

Vuelve a mirar la salida del paso anterior. Hay dos líneas que casi nadie lee, y son las que este laboratorio existe para enseñar. Esta es la primera.

```
Current partition replica assignment
{"version":1,"partitions":[... "partition":0,"replicas":[1,2,3] ...]}

Save this to use as the --reassignment-json-file option during rollback
```

ESO ES LA FOTO DE CÓMO ESTABA TODO ANTES

Y viene **en el formato exacto que el propio comando acepta**. Kafka te está diciendo, con todas las letras, que lo guardes.

Es lo único que te permite deshacer esta operación. Si el plan te deja el clúster peor, y pasa, porque un plan automático no sabe de tus racks ni de tus discos ni de cuál es la partición caliente, la vuelta atrás es aplicar ese JSON. Si no lo guardaste, tienes que reconstruirlo a mano, partición por partición, desde una foto que ya no existe.

En este laboratorio el script te lo guarda solo

```
ls -l /tmp/lab08-rollback-novatech.lab08.pedidos.json
```

LO QUE SALE

```
-rw-r--r-- 1 rodrigo wheel 911 Aug 27 00:06
/tmp/lab08-rollback-novatech.lab08.pedidos.json
```

En tu servidor no hay quien te lo guarde. Se copia y se pega en un archivo **antes** de que la consola lo tape, y en una operación de verdad se manda por correo al equipo antes de tocar nada.

Y ahora deshazlo de verdad

Es el ejercicio que convierte esto en un músculo. El plan guardado es aplicable tal cual:

```
docker cp /tmp/lab08-rollback-novatech.lab08.pedidos.json \
  kafka-broker-1:/tmp/rollback.json

docker exec kafka-broker-1 kafka-reassign-partitions \
  --bootstrap-server kafka-broker-1:29092 \
  --reassignment-json-file /tmp/rollback.json --execute

docker exec kafka-broker-1 kafka-reassign-partitions \
  --bootstrap-server kafka-broker-1:29092 \
  --reassignment-json-file /tmp/rollback.json --verify
```

`docker cp <origen> <contenedor>:<destino>` — copia un archivo de tu máquina al interior del contenedor. **El comando de Kafka se ejecuta dentro, así que el archivo también tiene que estar dentro**

`--reassignment-json-file /tmp/rollback.json` — el plan que se va a aplicar, que aquí es el de vuelta

`sin --throttle` — a propósito, para que el paso 6 tenga con qué comparar

LO LOGRASTE SI...

Tras el `--verify`, la foto del tópico vuelve a ser **exactamente la del paso 1**, réplica por réplica y en el mismo orden. Compruébalo:

```
kafka-cli/describe-topic.sh novatech.lab08.pedidos
```

Y UNA ADVERTENCIA QUE VALE EL PASO ENTERO

Deshacer no es gratis. Es otra reasignación completa, con su propia copia de datos, su propio impacto y su propio plan de vuelta. **«Puedo revertirlo» no significa «puedo revertirlo sin consecuencias».**

Los frenos que quedan puestos

Qué vamos a hacer

Esta es la otra línea que casi nadie lee.

```
Warning: You must run --verify periodically, until the reassignment completes,
to ensure the throttle is removed.
The inter-broker throttle limit was set to 8000000 B/s
```

El `--throttle` le pone un techo a la copia para que no se coma el ancho de banda de tus clientes, y eso es lo correcto. **Pero ese techo no es un parámetro del comando.** Son configuraciones dinámicas que Kafka escribe en el clúster, y que se quedan ahí.

Vuelve a reasignar, con throttle, y míralas

```
kafka-cli/reassign-partitions.sh novatech.lab08.pedidos 1,2,3,4 8000000
kafka-cli/describe-broker-config.sh 1 | grep throttled
```

`kafka-cli/describe-broker-config.sh 1` — la configuración efectiva del broker 1, la misma herramienta del laboratorio 03

| `grep throttled` — filtra las líneas de los frenos. **Si no devuelve nada, quedó limpio**

LO QUE SALE MIENTRAS LA COPIA CORRE

```
follower.replication.throttled.rate=8000000 sensitive=false
synonyms={DYNAMIC_BROKER_CONFIG:follower.replication.throttled.rate=8000000}
leader.replication.throttled.rate=8000000 sensitive=false
synonyms={DYNAMIC_BROKER_CONFIG:leader.replication.throttled.rate=8000000}
```

Ese `DYNAMIC_BROKER_CONFIG` es el mismo origen que estudiaste en el laboratorio 03. **Alguien lo escribió en caliente**, y ese alguien fue el `--execute`.

Y aquí está la trampa, medida

Quien quita los frenos es el `--verify`, y **solo cuando todas las particiones terminaron.** Compruébalo tú mismo. Corre un `--verify` **enseguida**, mientras todavía queda copia pendiente:

```
docker exec kafka-broker-1 kafka-reassign-partitions \
  --bootstrap-server kafka-broker-1:29092 \
  --reassignment-json-file /tmp/plan.json --verify

echo "codigo de salida: $?"
kafka-cli/describe-broker-config.sh 1 | grep throttled
```

LO QUE SALIÓ EN LA CORRIDA MEDIDA

```
Reassignment of partition novatech.lab08.pedidos-0 is still in progress.
... (5 de 6 particiones en curso)

codigo de salida: 0

follower.replication.throttled.rate=8000000 ...
leader.replication.throttled.rate=8000000 ...
```

LÉELO DESPACIO, PORQUE ES EL HALLAZGO DEL LABORATORIO

Quedaban **cinco particiones copiando**. El comando dijo `still in progress`, **no limpió nada**, y devolvió **código de salida 0**.

Un script que corre `--verify` una sola vez y mira el código de salida **se da por satisfecho y deja el clúster con los frenos puestos**, indefinidamente. Meses después alguien investiga por qué la replicación va lenta y encuentra un `throttled.rate` que nadie recuerda haber puesto.

Ahora hazlo bien

```
docker exec kafka-broker-1 kafka-reassign-partitions \
  --bootstrap-server kafka-broker-1:29092 \
  --reassignment-json-file /tmp/plan.json --verify
```

Repítelo hasta que **ninguna partición diga** `in progress`.

CUANDO POR FIN TERMINAN TODAS

```
Reassignment of partition novatech.lab08.pedidos-0 is completed.
Reassignment of partition novatech.lab08.pedidos-1 is completed.
Reassignment of partition novatech.lab08.pedidos-2 is completed.
Reassignment of partition novatech.lab08.pedidos-3 is completed.
Reassignment of partition novatech.lab08.pedidos-4 is completed.
Reassignment of partition novatech.lab08.pedidos-5 is completed.

Clearing broker-level throttles on brokers 1,2,3,4
Clearing topic-level throttles on topic novatech.lab08.pedidos
```

```
kafka-cli/describe-broker-config.sh 1 | grep throttled
```

Y AHORA SÍ

(no devuelve nada)

LO LOGRASTE SI...

Viste los frenos puestos, viste que un `--verify` prematuro **no los quita y aun así devuelve o**, y viste el `Clearing` del `--verify` final.

Y el `grep throttled` final **no devuelve nada**.

UN DETALLE QUE ENGAÑA, Y HAY QUE CONOCERLO

Si corres el `--execute` **sin** `--throttle`, Kafka no pone ningún freno. Pero el `--verify` imprime `Clearing ... throttles` **de todos modos**. Ese mensaje **no es prueba de que hubiera throttles**. De ahí sale la creencia de que Kafka los pone solo. No los pone. **Los pones tú**.

«Sin detener nada» no es «sin que nadie lo note»

Qué vamos a hacer

Vuelve a la terminal P y espera a que el productor termine. Su línea final es la que cierra el laboratorio.

LO QUE SALIÓ EN LA CORRIDA MEDIDA, CON LA REASIGNACIÓN ENCIMA

```
300000 records sent, 4998.916901 records/sec (1.22 MB/sec), 9.11 ms avg latency,
2016.00 ms max latency, 6 ms 50th, 10 ms 95th, 12 ms 99th, 1611 ms 99.9th.
```

Cómo se lee, y son dos lecturas opuestas

Métrica	Valor	Qué dice
Mensajes enviados	300 000 de 300 000	Cero pérdidas
Errores fatales	0	La operación fue transparente para la aplicación
Latencia media	9,11 ms	Un tablero de promedios diría que no pasó nada
Latencia p50	6 ms	La mitad de los mensajes ni se enteró
Latencia p95	10 ms	El 95 % tampoco
Latencia p99	12 ms	El 99 % tampoco
Latencia p99.9	1 611 ms	Uno de cada mil tardó más de un segundo y medio
Latencia máxima	2 016 ms	El peor tardó dos segundos

MIRA EL SALTO ENTRE EL P99 Y EL P99.9

De 12 ms a 1 611 ms. Ciento treinta y cuatro veces, en una décima de punto porcentual de los mensajes.

Es exactamente la lección del laboratorio 07, aplicada a una operación en vez de a un benchmark.

El promedio dice que no pasó nada y el percentil dice que sí. Si tu tablero solo muestra promedios, esta operación se ve perfecta.

SOBRE LOS WARN DEL PRODUCTOR

La documentación del curso menciona que durante una reasignación el productor puede registrar avisos de `NOT_LEADER_OR_FOLLOWER`. Ocurre cuando el liderazgo de una partición se mueve y el cliente le sigue escribiendo al broker viejo durante unos milisegundos.

En las dos corridas con las que se midió esta guía no apareció ninguno, y aun así la cola de latencia se disparó. **Si a ti tampoco te sale, no significa que la operación fuera gratis.** El rastro fiable es el p99.9, no el log.

LO LOGRASTE SI...

Puedes decir las dos cosas en la misma frase sin contradecirte. **No se perdió ni un mensaje, y la operación se notó.**

Si tu p99.9 salió parecido al p99, es que tu reasignación terminó antes de que el productor arrancara de verdad. Repite el paso 2 con más mensajes y el paso 4 encima.

QUÉ SIGNIFICA ESTO PARA SUNAT

Una reasignación es transparente para la **aplicación**, si los clientes tienen reintentos configurados. **No es transparente para un acuerdo de nivel de servicio escrito en percentiles.**

Por eso se hace fuera de hora punta y con `--throttle`. Y por eso al equipo se le avisa antes, no después.

Lo que aprendiste

1 Agregar capacidad y usar capacidad son dos operaciones.

Un broker nuevo entra vacío y se queda vacío. Si nadie reasigna, pagaste un servidor para nada. Y al revés, si alguien reasigna sin avisar, se está moviendo tráfico de producción por la red.

2 El plan de vuelta se guarda antes de ejecutar.

Lo imprime el `--execute` y se lo lleva el scroll. Es lo único que permite deshacer. En una operación de verdad se guarda en un archivo con nombre y fecha, y se manda al equipo **antes** de tocar nada.

3 `--verify` se corre en bucle, hasta que ninguna partición diga `in progress`.

Es la fase que limpia. Correrla una sola vez deja el clúster con límites de replicación puestos que nadie va a recordar, **y encima devuelve código de salida 0**, así que ni un script se entera. Comprueba después que se fueron.

4 «Sin downtime» se afirma con percentiles, no con promedios.

No se perdió un mensaje, y el p99.9 pasó de milisegundos a más de un segundo y medio. Las dos cosas son verdad, y la segunda es la que el usuario nota.

LA LECTURA QUE SE USA PARA TODO

Toda operación en caliente deja dos rastros. Una forma de deshacerla, y algo encendido que hay que apagar. Si no sabes cuáles son, no terminaste la operación.

Para profundizar

Lo que sigue quedó fuera del recorrido por tiempo. **Todos estos comandos se ejecutaron contra este mismo clúster.**

A · Drenar el broker y sacarlo

Pasó el pico y hay que devolver el servidor. **Apagarlo en seco pierde las réplicas que tiene**, así que primero se mueven a los que se quedan.

```
kafka-cli/drain-broker.sh novatech.lab08.pedidos 1,2,3
kafka-cli/describe-topic.sh novatech.lab08.pedidos
```

1, 2, 3 — la lista de los que se quedan. El 4 no está, así que sus réplicas se van a los otros tres. Es el mismo comando del paso 4, con otra lista

Lo que hay que mirar. Tras el drenaje, **ni un 4 en toda la columna Replicas**. Ahora sí se puede apagar sin perder nada:

```
docker compose -f infra/docker-compose.yml --profile scale stop kafka-broker-4
kafka-cli/list-brokers.sh
```

El clúster sigue operativo con 3 brokers y **el quórum no se enteró**, porque el 4 nunca fue *voter*. Es la contraparte exacta del paso 3.

B · Configuración dinámica del broker

```
kafka-cli/describe-broker-config.sh 1
kafka-cli/alter-broker-config.sh 1 num.replica.fetchers=2
kafka-cli/describe-broker-config.sh 1 | grep num.replica.fetchers
```

LO QUE SALE

```
num.replica.fetchers=2 sensitive=false
synonyms={DYNAMIC_BROKER_CONFIG:num.replica.fetchers=2,
          DEFAULT_CONFIG:num.replica.fetchers=1}
```

Lo que hay que mirar. Aparecen **los dos orígenes**. El `DEFAULT_CONFIG` de fábrica sigue ahí abajo y el `DYNAMIC_BROKER_CONFIG` le gana encima. **Es exactamente lo que viste en el laboratorio 03**, aplicado a un parámetro que sí sirve durante una reasignación, porque controla cuántos hilos copian réplicas.

C · Limpiar los frenos a mano

Si te quedaste con throttles puestos, esta es la salida:

```
docker exec kafka-broker-1 kafka-configs \
  --bootstrap-server kafka-broker-1:29092 \
  --alter --entity-type brokers --entity-name 1 \
  --delete-config leader.replication.throttled.rate,follower.replication.throttled.rate
```

--entity-type brokers --entity-name 1 — sobre qué broker. **Hay que repetirlo por cada broker** del clúster

--delete-config — borra esas configuraciones dinámicas y las devuelve a su valor de fábrica

Y comprueba con `kafka-cli/describe-broker-config.sh 1 | grep throttled` que ya no queda nada.

D · El ciclo completo, con tráfico

Es la prueba de fuego, y el entregable del laboratorio. Con un productor corriendo en segundo plano, en este orden:

#	Paso
1	Agrega el broker 4
2	Reasigna a los 4 brokers, con <code>--throttle</code>
3	Guarda el plan de vuelta en un archivo con nombre y fecha
4	Cambia <code>num.replica.fetchers=2</code> en caliente en el broker 4
5	Drena el broker 4 y apágalo
6	Comprueba que no quedó ningún <code>throttled</code> en ningún broker

Documenta cuatro cosas. ¿Hubo pérdida de mensajes? ¿Cuál fue el paso más lento? ¿Cuánto se movió el p99.9 del productor? ¿El clúster quedó limpio?

E · El reporte del laboratorio

Está en `plantillas/reporte-entregable.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

Corta el productor de la terminal P con `Ctrl+C` si sigue corriendo. Después, desde la terminal T:

```
bin/stop-lab.sh
```

Ese comando apaga los contenedores y conserva los volúmenes, incluido el del broker 4. Si quieres empezar de cero la próxima vez, el que borra los datos es `bin/reset-lab.sh`.

Comprueba con `docker ps --filter name=kafka-broker` que no quede nada corriendo.

LO QUE TE LLEVAS

La próxima vez que alguien diga «agregamos un servidor al clúster», la primera pregunta ya no es «¿está arriba?». Es **«¿le movieron carga, guardaron el plan de vuelta, y comprobaron que no quedó nada encendido?»**.